

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

ACTIVIDAD LABORATORIO NO.2
ACTIVIDAD: ALGORITMO PARA BÚSQUEDA DE RAÍCES REALES

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING HERNAN ALONSO MANCIPE BOHORQUEZ
PROFESOR

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
FACULTAD DE INGENIERÍA – INGENIERIA INFORMÁTICA
BOGOTÁ D.C.
02 DE JUNIO
2025

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Tabla de Contenido

RESUMEN	3
INTRODUCCIÓN	3
DESARROLLO DEL LABORATORIO	4
FUNCIONES	4
Funciones principales:	4
Derivadas parciales:	4
Otras funciones implementadas:	4
CONDICION INICIAL	5
CODIGO COMPLETO	5
Código a copiar y pegar en Google Colab	5
Escalabilidad del imágenes del código fuente:	7
SALIDA DEL CODIGO	10
Imagen de tabla de resultados	11
Imagen resultados, solución final y verificación $f(x,y)$ y $g(x,y)$	11
Imagen grafico convergencia	11
INDICACIONES ADICIONALES	12
CONCLUSIÓN	12
BIBLIOGRAFÍA	13

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

RESUMEN

Este laboratorio me permitió fortalecer mis competencias en el diseño y análisis de algoritmos, pruebas de escritorio y programación, especialmente en la implementación de métodos numéricos para resolver sistemas de ecuaciones no lineales. En esta ocasión, trabajé con Python, utilizando bibliotecas como NumPy, pandas y matplotlib, que facilitaron tanto el desarrollo computacional como la visualización de los resultados.

Inicialmente, planteé un sistema de ecuaciones no lineales compuesto por dos funciones:

- $f(x, y) = x^2 - y - 2$
- $g(x, y) = x + y^2 - 2$

Estas funciones fueron utilizadas para aplicar el método de Newton-Raphson para sistemas, que requiere el cálculo de derivadas parciales, la construcción de la matriz Jacobiana y su inversa en cada iteración. Este procedimiento iterativo permitió aproximar una solución numérica al sistema planteado, partiendo de una condición inicial determinada.

Durante el desarrollo, implementé un código estructurado que calcula los valores de cada función y sus derivadas, construye la Jacobiana, evalúa el vector $f(x, y)$, y actualiza las soluciones usando el producto de la inversa de la Jacobiana por F . Además, se calculó el error relativo en cada paso para verificar la convergencia del método, permitiendo detener el proceso cuando se alcanzó una tolerancia previamente definida.

Los resultados obtenidos en cada iteración fueron almacenados en una tabla con pandas, permitiendo visualizar de manera ordenada valores como los componentes de la Jacobiana, el vector F , las soluciones sucesivas y el error relativo. Posteriormente, estos datos fueron exportados a un archivo .csv para su análisis externo.

Adicionalmente, desarrollé una gráfica de convergencia con matplotlib, la cual muestra cómo evolucionan los valores de x e y a lo largo de las iteraciones. Esta visualización resultó clave para validar gráficamente la estabilidad y efectividad del método aplicado.

Este laboratorio no solo me permitió resolver un sistema de ecuaciones no lineales mediante un enfoque numérico, sino también integrar diversas herramientas computacionales para automatizar el proceso, analizar resultados y visualizar el comportamiento del algoritmo. Reafirmé la utilidad de la programación como medio para resolver problemas matemáticos complejos, así como la importancia de una buena estructura algorítmica y documentación para interpretar y validar los resultados obtenidos.

INTRODUCCIÓN

En el marco del estudio de algoritmos y métodos numéricos, este laboratorio tuvo como propósito fortalecer y aplicar conocimientos orientados a la resolución de sistemas de ecuaciones no lineales, mediante la implementación computacional del método de Newton-Raphson para sistemas. Para ello, utilicé el lenguaje Python junto con bibliotecas como NumPy, Pandas y Matplotlib, lo que me permitió automatizar los cálculos, analizar la convergencia iterativa y visualizar los resultados de forma estructurada y precisa.

Inicialmente, definí las funciones no lineales $f(x, y) = x^2 - y - 2$ y $g(x, y) = x + y^2 - 2$ junto con sus derivadas parciales, con el fin de construir la matriz Jacobiana y su inversa en cada iteración del algoritmo. A partir de un valor inicial dado, ejecuté el proceso iterativo controlando parámetros como la tolerancia y el número máximo de iteraciones, con el objetivo de garantizar la precisión y estabilidad del método.

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Durante el desarrollo del código, registré detalladamente cada paso del algoritmo: los valores intermedios de x , y , los valores de las funciones, la matriz Jacobiana, su inversa, el producto con el vector $f(x, y)$, y el error relativo en cada iteración. Esta información fue almacenada en un DataFrame y exportada como archivo CSV para su análisis. Además, generé una gráfica de convergencia para visualizar el comportamiento de las soluciones iterativas.

Esta actividad no solo permitió encontrar una solución numérica aproximada al sistema planteado, sino también integrar múltiples herramientas computacionales para estructurar el proceso de resolución, validar resultados y representar gráficamente la evolución del algoritmo. Finalmente, el laboratorio reforzó competencias clave como el diseño de algoritmos, la depuración por pruebas de escritorio, y el análisis de errores, pilares fundamentales en la solución de problemas reales mediante programación científica.

DESARROLLO DEL LABORATORIO

Para iniciar este laboratorio de métodos numéricos, se planteó un escenario en el que fenómenos como el crecimiento de amenazas o el aumento de la carga en un sistema informático pueden presentar comportamientos no lineales. En este sentido, las ecuaciones cuadráticas se utilizan como modelo para representar dichos crecimientos, ya que sus raíces permiten identificar puntos críticos en los que el sistema alcanza niveles significativos de carga, rendimiento o vulnerabilidad. Estos puntos críticos pueden interpretarse como eventos clave donde un sistema podría colapsar, perder estabilidad o volverse más susceptible a amenazas. Detectarlos con anticipación es fundamental para determinar cuándo intervenir y así evitar fallos o brechas de seguridad. A continuación, se presenta el código completo desarrollado durante el laboratorio, junto con su imagen de salida, lo cual permite observar y validar gráficamente los resultados obtenidos mediante los métodos aplicados.

FUNCIONES

Las funciones implementadas en el código son las siguientes, según el propio código y la salida impresa:

Funciones principales:

- $f(x, y) = x^2 - y - 2$
- $g(x, y) = x + y^2 - 2$

Derivadas parciales:

- $df/dx = 2x$
- $df/dy = -1$
- $dg/dx = 1$
- $dg/dy = 2y$

Otras funciones implementadas:

- **jacobian(x, y)**: Calcula la matriz Jacobiana con las derivadas parciales.
- **inverse_jacobian(x, y)**: Calcula la inversa de la Jacobiana, verificando si el determinante es no singular.
- **f(x, y)**: Devuelve el vector $[f(x, y), g(x, y)]$.
- **newton_raphson(x0, y0, tol, max_iter)**: Implementa el método de Newton-Raphson para sistemas no lineales, iterando hasta alcanzar la tolerancia o el máximo de iteraciones.
- Estas funciones se utilizan para resolver el sistema no lineal $f(x, y) = 0$ y $g(x, y) = 0$.

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

CONDICION INICIAL

La condición inicial (especificada en el código):

- $x_0 = 1.5$
- $y_0 = 0.5$

CODIGO COMPLETO

A continuación, remito el código matriz completo en este desarrollo de este algoritmo, el cual se puede copiar y pegar en Google Colab:

[Código a copiar y pegar en Google Colab](#)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Funciones f(x, y) y g(x, y)
def f(x, y):
    return x**2 - y - 2

def g(x, y):
    return x + y**2 - 2

# Derivadas parciales
def df_dx(x, y):
    return 2 * x

def df_dy(x, y):
    return -1

def dg_dx(x, y):
    return 1

def dg_dy(x, y):
    return 2 * y

# Jacobiana
def jacobian(x, y):
    return np.array([[df_dx(x, y), df_dy(x, y)], [dg_dx(x, y), dg_dy(x, y)]])

# Inversa de la Jacobiana
def inverse_jacobian(x, y):
    J = jacobian(x, y)
    det = np.linalg.det(J)
    if abs(det) < 1e-10:
        print(f"Jacobiana singular (det = {det:.5e}) en x = {x:.5f}, y = {y:.5f}")
        return None
    return np.linalg.inv(J)

# Vector F(x, y)
def F(x, y):
    return np.array([f(x, y), g(x, y)])

# Método de Newton-Raphson para sistemas
def newton_raphson(x0, y0, tol, max_iter):
```

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```

resultados = []
x_n = np.array([x0, y0], dtype=float)

for i in range(max_iter):
    f_val = f(x_n[0], x_n[1])
    g_val = g(x_n[0], x_n[1])
    F_val = F(x_n[0], x_n[1])
    J_inv = inverse_jacobian(x_n[0], x_n[1])
    if J_inv is None:
        print(f"Detención en iteración {i+1} por Jacobiana singular")
        break
    x_n1 = x_n - J_inv @ F_val
    e_r = np.linalg.norm(x_n1 - x_n) / np.linalg.norm(x_n1) if np.linalg.norm(x_n1) != 0
else float('inf')

# Formatear valores pequeños en notación científica
f_val_str = f"{f_val:.5e}" if abs(f_val) < 0.001 else f"{f_val:.5f}"
g_val_str = f"{g_val:.5e}" if abs(g_val) < 0.001 else f"{g_val:.5f}"
F_str = [f"{v:.5e}" if abs(v) < 0.001 else f"{v:.5f}" for v in F_val]
prod_str = [f"{v:.5e}" if abs(v) < 0.001 else f"{v:.5f}" for v in (J_inv @ F_val)]

resultados.append({
    'Iteración': i + 1,
    'x': f"{x_n[0]:.5f}",
    'y': f"{x_n[1]:.5f}",
    'f(x,y)': f_val_str,
    'g(x,y)': g_val_str,
    'df/dx': f"{df_dx(x_n[0], x_n[1]):.5f}",
    'df/dy': f"{df_dy(x_n[0], x_n[1]):.5f}",
    'dg/dx': f"{dg_dx(x_n[0], x_n[1]):.5f}",
    'dg/dy': f"{dg_dy(x_n[0], x_n[1]):.5f}",
    'Jacobian': [[f"{v:.5f}" for v in row] for row in jacobian(x_n[0], x_n[1]).round(5)],
    'Inversa Jacobiana': [[f"{v:.5f}" for v in row] for row in J_inv.round(5)] if J_inv is
not None else "Singular",
    'F': F_str,
    'Producto': prod_str,
    'x (i-1)': [f"{v:.5f}" for v in x_n.round(5)],
    'x_i': [f"{v:.5f}" for v in x_n1.round(5)],
    'e_r': f"{e_r:.6f}"
})

if e_r < tol:
    break
x_n = x_n1

return resultados, x_n1

# Parámetros
x0, y0 = 1.5, 0.5 # Condición inicial
tolerancia = 0.002
max_iteraciones = 20

# Ejecutar Newton-Raphson
resultados, sol_final = newton_raphson(x0, y0, tolerancia, max_iteraciones)

# Crear y mostrar tabla con formato mejorado

```

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```

df = pd.DataFrame(resultados)
print("\nTabla de resultados (x_0 = 1.5, y_0 = 0.5):")
pd.set_option('display.colheader_justify', 'center')
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
print(df.to_string(index=False, justify='center'))

# Guardar tabla en CSV
df.to_csv('resultados_newton_sistema.csv', index=False)
print("\nResultados guardados en 'resultados_newton_sistema.csv'")

# Verificar solución
print(f"\nSolución final: x = {sol_final[0]:.5f}, y = {sol_final[1]:.5f}")
print(f"Verificación f(x, y) = {f(sol_final[0], sol_final[1]):.5f} (debería ser cercano a 0)")
print(f"Verificación g(x, y) = {g(sol_final[0], sol_final[1]):.5f} (debería ser cercano a 0)")

# Funciones trabajadas
print("\nFunciones trabajadas:")
print("f(x, y) = x^2 - y - 2")
print("g(x, y) = x + y^2 - 2")
print()

# Gráfica de convergencia con matplotlib
iterations = [res['Iteración'] for res in resultados]
x_values = [float(res['x']) for res in resultados]
y_values = [float(res['y']) for res in resultados]

plt.figure(figsize=(8, 6))
plt.plot(iterations, x_values, label='x', color='#1E90FF', marker='o')
plt.plot(iterations, y_values, label='y', color='#FF4500', marker='o')
plt.xlabel('Iteración')
plt.ylabel('Valor')
plt.title('Convergencia de x e y en el método de Newton-Raphson')
plt.legend()
plt.grid(True)
plt.xticks(iterations)
plt.ylim(0.4, 1.7)
plt.savefig('convergencia_newton.png')
plt.show()
print("\nGráfica guardada como 'convergencia_newton.png'")

```

Escalabilidad del imágenes del código fuente:

A continuación, las imágenes del código para generar escalabilidad (adaptación en el programa);

Primera parte imagen del código fuente:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Funciones f(x, y) y g(x, y)
def f(x, y):
    return x**2 - y - 2

def g(x, y):
    return x + y**2 - 2

# Derivadas parciales
def df_dx(x, y):
    return 2 * x

def df_dy(x, y):
    return -1

def dg_dx(x, y):
    return 1

def dg_dy(x, y):
    return 2 * y

# Jacobiana
def jacobian(x, y):
    return np.array([[df_dx(x, y), df_dy(x, y)], [dg_dx(x, y), dg_dy(x, y)]])

# Inversa de la Jacobiana
def inverse_jacobian(x, y):
    J = jacobian(x, y)
    det = np.linalg.det(J)
    if abs(det) < 1e-10:
        print(f"Jacobiana singular (det = {det:.5e}) en x = {x:.5f}, y = {y:.5f}")
        return None
    return np.linalg.inv(J)

# Vector F(x, y)
def F(x, y):
    return np.array([f(x, y), g(x, y)])

```

Segunda parte imagen del código fuente:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```
# Método de Newton-Raphson para sistemas
def newton_raphson(x0, y0, tol, max_iter):
    resultados = []
    x_n = np.array([x0, y0], dtype=float)

    for i in range(max_iter):
        f_val = f(x_n[0], x_n[1])
        g_val = g(x_n[0], x_n[1])
        F_val = [f_val, g_val]
        J_inv = inverse_jacobian(x_n[0], x_n[1])
        if J_inv is None:
            print(f"Detención en iteración {i+1} por Jacobiana singular")
            break
        x_n1 = x_n - J_inv @ F_val
        e_r = np.linalg.norm(x_n1 - x_n) / np.linalg.norm(x_n1) if np.linalg.norm(x_n1) != 0 else float('inf')

        # Formatear valores pequeños en notación científica
        f_val_str = f"{f_val:.5e}" if abs(f_val) < 0.001 else f"{f_val:.5f}"
        g_val_str = f"{g_val:.5e}" if abs(g_val) < 0.001 else f"{g_val:.5f}"
        F_str = [f"{v:.5e}" if abs(v) < 0.001 else f"{v:.5f}" for v in F_val]
        prod_str = [f"{v:.5e}" if abs(v) < 0.001 else f"{v:.5f}" for v in (J_inv @ F_val)]

        resultados.append({
            'Iteración': i + 1,
            'x': f"{x_n[0]:.5f}",
            'y': f"{x_n[1]:.5f}",
            'f(x,y)': f_val_str,
            'g(x,y)': g_val_str,
            'df/dx': f"{df_dx(x_n[0], x_n[1]):.5f}",
            'df/dy': f"{df_dy(x_n[0], x_n[1]):.5f}",
            'dg/dx': f"{dg_dx(x_n[0], x_n[1]):.5f}",
            'dg/dy': f"{dg_dy(x_n[0], x_n[1]):.5f}",
            'Jacobiana': [[f"{v:.5f}" for v in row] for row in jacobian(x_n[0], x_n[1]).round(5)],
            'Inversa Jacobiana': [[f"{v:.5f}" for v in row] for row in J_inv.round(5)] if J_inv is not None else "Singular",
            'F': F_str,
            'Producto': prod_str,
            'x_(i-1)': [f"{v:.5f}" for v in x_n.round(5)],
            'x_i': [f"{v:.5f}" for v in x_n1.round(5)],
            'e_r': f"{e_r:.6f}"
        })

    if e_r < tol:
        break
    x_n = x_n1

    return resultados, x_n1
```

Tercera parte imagen del código fuente:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```

# Parámetros
x0, y0 = 1.5, 0.5 # Condición inicial
tolerancia = 0.002
max_iteraciones = 20

# Ejecutar Newton-Raphson
resultados, sol_final = newton_raphson(x0, y0, tolerancia, max_iteraciones)

# Crear y mostrar tabla con formato mejorado
df = pd.DataFrame(resultados)
print("\nTabla de resultados (x_0 = 1.5, y_0 = 0.5):")
pd.set_option('display.colheader_justify', 'center')
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
print(df.to_string(index=False, justify='center'))

# Guardar tabla en CSV
df.to_csv('resultados_newton_sistema.csv', index=False)
print("\nResultados guardados en 'resultados_newton_sistema.csv'")

# Verificar solución
print(f"\nSolución final: x = {sol_final[0]:.5f}, y = {sol_final[1]:.5f}")
print(f"Verificación f(x, y) = {f(sol_final[0], sol_final[1]):.5f} (debería ser cercano a 0)")
print(f"Verificación g(x, y) = {g(sol_final[0], sol_final[1]):.5f} (debería ser cercano a 0)")

# Funciones trabajadas
print("\nFunciones trabajadas:")
print("f(x, y) = x^2 - y - 2")
print("g(x, y) = x + y^2 - 2")
print()

# Gráfica de convergencia con matplotlib
iterations = [res['Iteración'] for res in resultados]
x_values = [float(res['x']) for res in resultados]
y_values = [float(res['y']) for res in resultados]

plt.figure(figsize=(8, 6))
plt.plot(iterations, x_values, label='x', color='#1E90FF', marker='o')
plt.plot(iterations, y_values, label='y', color='#FF4500', marker='o')
plt.xlabel('Iteración')
plt.ylabel('Valor')
plt.title('Convergencia de x e y en el método de Newton-Raphson')
plt.legend()
plt.grid(True)
plt.xticks(iterations)
plt.ylim(0.4, 1.7)
plt.savefig('convergencia_newton.png')
plt.show()
print("\nGráfica guardada como 'convergencia_newton.png'")

```

SALIDA DEL CODIGO

La salida del código incluye varios elementos, que se pueden resumir de la siguiente manera:

- El código genera una tabla (guardada en resultados_newton_sistema.csv) que muestra el progreso del método de Newton-Raphson en cada iteración. La tabla incluye:
 - **Iteración:** Número de iteraciones.

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

- **x, y:** Valores de x y y en cada iteración.
- **f(x, y), g(x, y):** Evaluación de las funciones.
- **Derivadas parciales:** $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial g}{\partial x}$, $\frac{\partial g}{\partial y}$.
- **Jacobiana:** Matriz Jacobiana en cada iteración.
- **Inversa Jacobiana:** Inversa de la Jacobiana (o "Singular" si el determinante es cercano a 0).
- **F:** Vector $[f(x, y), g(x, y)]$.
- **Producto:** Resultado de $J^{-1} \cdot F$.
- **x_(i-1), x_i:** Valores anteriores y actuales de x .
- **e_r:** Error relativo.

A continuación, muestro las imágenes de la salida del código implementado (la imagen de la tabla se puede ampliar y de igual manera al ejecutar el código en Google Colab se puede denotar con facilidad):

Imagen de tabla de resultados

Tabla de resultados (x_0 = 1.5, y_0 = 0.5):															
Iteración	x	y	f(x,y)	g(x,y)	df/dx	df/dy	dg/dx	dg/dy	Jacobiana	Inversa Jacobiana	F	Producto	x _(i-1)	x _i	e _r
1	1.50000	0.50000	-0.25000	-0.25000	1.00000	1.00000	1.00000	1.00000	[[1.00000, -1.00000], [1.00000, 1.00000]]	[[0.25000, 0.25000], [-0.25000, 0.75000]]	[-0.25000, -0.25000]	[-0.12500, -0.12500]	1.50000	0.50000	0.00000
2	1.62500	0.62500	0.01562	0.01562	1.25000	1.00000	1.25000	1.25000	[[1.25000, -1.00000], [1.00000, 1.25000]]	[[0.24603, 0.13753], [-0.13753, 0.64338]]	[0.01562, 0.01562]	[0.00094, 0.00094]	1.62500	0.62500	0.00578
3	1.61803	0.61803	4.82233e-05	4.82233e-05	1.25011	1.00000	1.25011	1.25011	[[1.25011, -1.00000], [1.00000, 1.25011]]	[[0.24723, 0.13909], [-0.13909, 0.64729]]	[4.82233e-05, 4.82233e-05]	[2.35666e-05, 2.35666e-05]	1.61803	0.61803	0.00003

Imagen resultados, solución final y verificación f(x,y) y g(x,y)

```

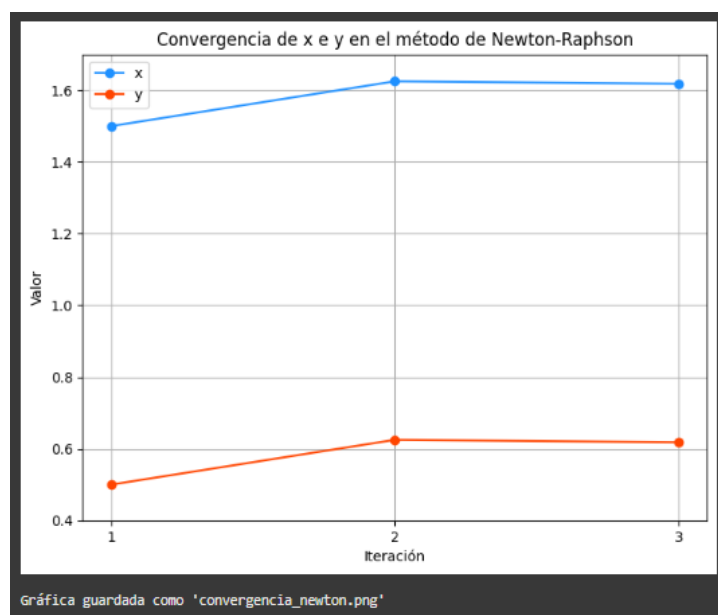
Resultados guardados en 'resultados_newton_sistema.csv'

Solución final: x = 1.61803, y = 0.61803
Verificación f(x, y) = 0.00000 (debería ser cercano a 0)
Verificación g(x, y) = 0.00000 (debería ser cercano a 0)

Funciones trabajadas:
f(x, y) = x^2 - y - 2
g(x, y) = x + y^2 - 2

```

Imagen grafico convergencia



Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

INDICACIONES ADICIONALES

A continuación, una serie de indicaciones que de igual manera considero son de gran relevancia frente a este desarrollo:

1. Los resultados se guardan en un archivo CSV llamado `resultados_newton_sistema.csv`
2. El método de Newton-Raphson itera hasta que el error relativo "**e_r**" sea menor que la tolerancia (0.002) o se alcancen las 20 iteraciones máximas.
3. Si la Jacobiana es singular (determinante cercano a 0), el código lo detecta e interrumpe la iteración.
4. El código genera una gráfica (guardada como `convergencia_newton.png`) que muestra la convergencia de los valores de x , y a lo largo de las iteraciones. La gráfica usa:
 - Eje x : Iteraciones.
 - Eje y : Valores de x (en azul) y y (en naranja).

Y con este resultado culmino el desarrollo de este trabajo, ¡¡¡mil gracias respetado y estimado, profesor Ing. Hernán Alonso!!!, muy valiosos sus conocimientos enseñados, estoy seguro me van a ayudar en mi futuro como ingeniero informático.

CONCLUSIÓN

El desarrollo de este laboratorio fue una oportunidad valiosa para integrar y aplicar diversos conceptos relacionados con algoritmos, métodos numéricos y programación estructurada. A través de la implementación directa del método de Newton-Raphson en Python, pude explorar en profundidad una técnica iterativa potente para la resolución de ecuaciones no lineales.

La codificación en Google Colab permitió estructurar paso a paso el algoritmo, definir la función, su derivada, y establecer condiciones como el valor inicial, la tolerancia y el número máximo de iteraciones. A medida que se ejecutaban las iteraciones, fue posible observar la convergencia hacia una raíz real de la ecuación, evaluando la eficacia del método en términos de precisión y velocidad de cálculo.

Durante el proceso, las pruebas de escritorio resultaron clave para validar el comportamiento del algoritmo antes de su ejecución completa. Esto facilitó la detección y corrección de errores relacionados con la inicialización de variables, el control de flujo y las condiciones de parada. Además, el uso de herramientas gráficas como Desmos permitió representar visualmente la función objetivo, sirviendo como mecanismo de validación y análisis del comportamiento de la raíz aproximada.

Este laboratorio no solo fortaleció mis habilidades en programación numérica, sino que también me permitió comprender cómo los métodos iterativos como Newton-Raphson pueden aplicarse eficazmente en la resolución de problemas matemáticos reales, incluyendo aquellos relacionados con la modelación de fenómenos no lineales en contextos como la ciberseguridad y la gestión de sistemas.

Agradezco la oportunidad de haber desarrollado esta práctica, ya que consolidó mi comprensión del enfoque numérico para la resolución de ecuaciones, destacando la importancia de una implementación cuidadosa, validada y analizada desde múltiples perspectivas.

En conclusión, este laboratorio fortaleció tanto mis competencias técnicas como mi capacidad de análisis y planificación para abordar problemas numéricos de forma estructurada, validada y visualmente interpretada. Agradezco profundamente la oportunidad de aplicar estos

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

conceptos de manera práctica, ya que representan una base sólida para resolver problemas complejos en el ámbito profesional.

Muchas gracias, profesor, por compartir estos conocimientos que sin duda serán fundamentales en mi formación como ingeniero informático.

BIBLIOGRAFÍA

Respetado profesor Ing. Hernán Alonso a continuación, la bibliografía implementada:

- ✓ Tema 1. Principios fundamentales del conteo. Tema 2. Análisis numérico y de los errores. Tema 3. Aritmética del computador. Tema 4. Cálculo de raíces e interpolación. Tema 5. Algoritmos de resolución y técnicas de aceleración,
- ✓ Burden, R. L., & Faires, J. D. (2011). Análisis Numérico (9.^a ed.). Cengage Learning.
- ✓ Referencia clave para la explicación y justificación del método de la secante.
- ✓ Sullivan, M. (2013). Álgebra y Trigonometría (9.^a ed.). Pearson.
- ✓ Para fundamentos sobre la resolución de ecuaciones cuadráticas
- ✓ Clases virtuales con el profesor Ing. Hernán Alonso Mancipe.
- ✓ Material de estudio del aula.